

Part 2 - Written Evaluation

1. Managing Manual QA + Automation QA

The right approach always depends on the SDLC and the process the project follows, but assuming a sprint-based workflow with defined user stories, I would structure the two roles as complementary, parallel lanes.

For every user story, the manual QA team is responsible for analyzing requirements, defining test cases, executing them, and reporting bugs against the functionality. Alongside this, each story carries a linked automation task that tracks the automation coverage for that same feature. In practice, automation runs approximately one sprint behind, it is difficult and often counterproductive to automate a feature before it is fully stable - so the automation engineer works from the previous sprint's accepted stories.

To ensure alignment, the automation QA reviews the manual test cases written by the manual team and syncs with them on any gaps or ambiguities before implementation begins. This keeps both sides working from the same source of truth and prevents duplicated or contradictory coverage.

Predictable delivery follows naturally from this structure: each sprint contains a defined set of manual tasks and a defined set of automation tasks, both estimated and tracked like any other development work. Clear ownership removes ambiguity, and the sprint scope is agreed at kick-off so there are no surprises at the end. To make this self-enforcing, I set up Jira automation rules that automatically create linked tasks for manual TC creation, manual execution, and automation coverage on every new story ticket. This means no story can be closed without those tasks being explicitly addressed, the process does not rely on people remembering to add QA work manually.

2. Inheriting an Existing Playwright Project

My first step is always to run the entire suite, observe what passes, what fails, and what is flaky, and build a clear baseline picture. I read through the tests alongside this to understand the structure, organization, and standards that have been followed, or not followed.

Once I have that overview, I categorize every failing or unstable test into one of three buckets: genuine product bugs (covered by existing bug reports or needing new ones), script issues (incorrect selectors, missing waits, outdated assertions), and truly flaky tests that need stabilization. I create tasks for each bucket, assign them, and prioritize the script fixes first since they produce false negatives and erode the team's trust in the suite.

With a reliable green/red signal established, I introduce standards in layers. Automated gates come first - ESLint, TypeScript strict mode, and PR review checklists, so that new bad code cannot enter without being caught. Alongside this I document the agreed guidelines explicitly: branch naming, commit message format, PR size limits, the maximum number of assertions per test, selector strategy, and so on. Written guidelines prevent the same conversations from happening repeatedly and give new team members a clear reference from day one. Structural improvements follow: reinforcing Page Object Model patterns, consolidating shared helpers, tagging tests for smoke vs regression runs, and improving the CI pipeline. Integrations and tooling enhancements (reporting, AI-assisted maintenance) come once the foundation is solid.

Long-term maintainability is the outcome of everything above, but it also depends on mindset. As a QA lead, part of the role is raising quality and testing awareness across the entire team, not just within QA. The end goal is a shift-left approach, which ultimately starts with the developers: their unit and integration tests, the dev testing they perform before handing off, and the quality of their PR reviews. When the whole team shares ownership of quality, the automation suite stays relevant and maintained rather than becoming an isolated QA artifact.

3. Working with Designers

Communication is the foundation. QA should be present in refinement and planning meetings alongside designers so that changes, new states, and edge cases are discussed before implementation rather than discovered during testing. Every story that touches the design should carry a QA task so nothing slips through unreviewed.

The Figma file must be kept up to date and treated as the single source of truth for expected visual behavior. When QA and designers are aligned on what "correct" looks like before development begins, the number of late-stage misunderstandings drops significantly. On the process side, the Jira automation mentioned above also applies here: if every design change is tracked as a ticket, and it should be - the automation rule guarantees that a QA task is created automatically. Nothing can pass by unreviewed simply because someone forgot to add a QA step.

For design-heavy pages where visual precision matters, I introduce snapshot and pixel-comparison tooling, such as Percy or Playwright's built-in visual comparisons integrated into the CI pipeline. This removes the reliance on human eyes catching every pixel-level regression and gives designers confidence that their intent is preserved across deployments.

4. Challenging Situation

What happened: I was responsible for an automated regression suite of over 50 test cases. The team requested that the suite run on every merge to the development branch and that any failure would automatically block the release to production. The additional constraint was that the entire execution had to complete in under 10 minutes to avoid becoming a bottleneck in the deployment pipeline.

What I decided: I started with a thorough analysis rather than jumping to optimization. Together with the development team, I reviewed which tests were genuinely necessary at the UI level and which could be replaced by existing integration tests already in the codebase. Several UI tests were redundant given lower-level coverage and were removed. For the remaining tests, I focused on stabilization first, reducing flakiness by reviewing each step for unnecessary rigidity, and merging tests that shared repeated setup steps into consolidated scenarios. Where tests required complex preconditions such as a logged-in user or pre-seeded data, I replaced the UI setup with direct API calls, which significantly cut the per-test overhead. I also introduced session reuse so that the authentication flow only ran once per worker rather than before every single test, which alone removed a significant chunk of total execution time.

How I communicated: I shared a short written summary of the analysis with the team and stakeholders before making any changes, outlining which tests would be removed and why, and what the projected impact on run time would be. This prevented surprises and gave the team the chance to flag anything I might have missed. Progress was tracked in a shared document that everyone could see throughout the sprint.

What I learned: Test optimization is never purely about the tests themselves. The biggest gains came from understanding the broader picture - what coverage already existed at other levels, how the CI infrastructure was configured, which dependencies were being installed unnecessarily, and how parallelization could be improved. The experience reinforced that a QA lead needs to think across the entire delivery pipeline, not just the test scripts.